



南京大學

本科畢業論文

院 系 計算機學院

專 業 計算機科學與技術

題 目 一種基於大語言模型的代碼

提交信息自動生成技術研究

年 級 2021 學 號 211220073

學生姓名 張繼文

指導教師 徐鋒 職 稱 教授

提交日期 2025 年 6 月 6 日



南京大学本科毕业论文（设计） 诚信承诺书

本人郑重承诺：所呈交的毕业论文（设计）（题目：一种基于大语言模型的代码提交信息自动生成技术研究）是在指导教师的指导下严格按照学校和院系有关规定由本人独立完成的。本毕业论文（设计）中引用他人观点及参考资源的内容均已标注引用，如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文（设计）等违纪行为。

作者签名：张继文

学号：211220073

日期：2025.5.17

南京大学本科生毕业论文（设计、作品）中文摘要

题目：一种基于大语言模型的代码提交信息自动生成技术研究

院系：计算机学院

专业：计算机科学与技术

本科生姓名：张继文

指导教师（姓名、职称）：徐锋 教授

摘要：

代码提交信息是对代码变更的一种自然语言描述。简单易懂且准确的代码提交信息能够使开发者快速了解代码变更的内容及原因，对程序开发与维护有着至关重要的作用。在过去，开发者需要人工对代码变更内容、目的及影响进行书面解释。这不仅费时费力，而且还可能存在格式混乱、语义模糊等问题，为开发工作造成了很大的困扰。后来，产生了代码提交信息自动生成（Automatic Commit Message Generation，ACMG）技术，能够对代码提交信息进行自动化的生成。传统的ACMG技术总体上可分为：基于规则的方法、基于检索的方法、基于早期学习的方法、混合方法。上述方法存在代码变更内容和原因描述不准确、高度依赖已有提交信息数据集、可读性差等问题。随着大语言模型（LLMs）的兴起，基于LLMs的ACMG技术在预测准确率、效率、成本、可读性等方面均超过了上述方法。但是，目前采用的基于LLMs生成代码提交信息的技术仍存在部分生成的提交信息缺失变更内容或变更原因的问题。

为解决在ACMG领域存在的上述技术缺陷，本文提出了一种基于大语言模型的多agent方法（Large Language Model-Based Multi-Agent Approach, LLM-MAA），采用大语言模型进行检查反馈，用以生成更加准确的、更加规范的、满足指定要求的代码提交信息。在Python语言上，该方法取得了和基准方法（将代码变更（diff）与提示模版构成提示输入给大语言模型，令大语言模型生成代码提交信息）相比更优的评分结果。实验结果表明，本文所提方法较基准方法在测试数据集、基座模型均相同的情况下，效果更优。

关键词：代码提交信息自动生成；大语言模型；质量评价

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Research on Automatic Commit Message Generation Based on Large Language Models

DEPARTMENT: School of Computer Science

SPECIALIZATION: Computer Science and Technology

UNDERGRADUATE: Zhang Jiwen

MENTOR: Professor Xu Feng

ABSTRACT:

The code commit message is a natural language description of code changes. Clear, concise, and accurate commit messages enable developers to quickly understand the content and rationale behind code modifications, playing a crucial role in software development and maintenance. Traditionally, developers manually documented code changes, purposes, and impacts, which proved time-consuming, labor-intensive, and prone to inconsistent formatting or ambiguous semantics, creating significant challenges. Later, Automatic Commit Message Generation (ACMG) techniques emerged to automate this process. Conventional ACMG approaches include Rule-based Approaches, Retrieval-based Approaches, Learning-based Approaches, and Hybrid Approaches. These approaches suffer from inaccuracies in describing code changes and motivations, over-reliance on existing commit message datasets, and poor readability. With the rise of large language models (LLMs), LLM-based ACMG techniques have surpassed traditional approaches in prediction accuracy, efficiency, cost-effectiveness, and readability. However, current LLM-based approaches still occasionally generate commit messages that omit critical change details or underlying reasons.

To address these technical limitations in ACMG, this paper proposes a Large Language Model-Based Multi-Agent Approach (LLM-MAA) that incorporates a feedback mechanism utilizing large language models to generate more accurate, standardized, and requirement-compliant commit messages. For Python language implementations, our method demonstrates superior evaluation metrics compared to the baseline approach (which directly feeds code diffs with prompt templates to LLMs for message genera-

tion). Experimental results under identical test datasets and base models confirm that the proposed approach outperforms the baseline approach in effectiveness.

KEYWORDS: Automatic commit message generation; Large language models; Quality evaluation

目 录

目 录	VII
插图目录	XI
表格目录	XIII
第一章 导论	1
1.1 研究背景	1
1.1.1 代码提交信息	1
1.1.2 人工代码提交信息生成	1
1.1.3 自动化代码提交信息生成	3
1.2 研究现状	5
1.2.1 相关工作概述	5
1.2.2 现有工作的不足	6
1.3 研究思路	6
1.4 本文主要贡献	7
1.5 论文组织	7
第二章 相关工作	9
2.1 相关工作概述	9
2.2 基于规则的方法	9
2.3 基于检索的方法	10
2.4 基于学习的方法	10
2.5 混合方法	11
2.6 本章小结	12

第三章 基于大语言模型的代码提交信息自动生成方法	13
3.1 预备知识	13
3.1.1 自然语言处理 (NLP)	13
3.1.2 大语言模型	14
3.1.3 代码专用预训练模型	14
3.1.4 代码提交信息规范	15
3.2 基于大语言模型的多 agent 方法	15
3.2.1 总体流程	15
3.2.2 generator_agent 模块	16
3.2.3 reviewer_agent 模块	16
3.2.4 基座模型	17
3.2.5 结果处理	17
3.3 本章小结	18
第四章 实验与评估	19
4.1 实验环境	19
4.2 基准方法选取	19
4.3 测试集选取	20
4.3.1 测试集获取	20
4.3.2 测试集处理	20
4.4 基座模型选择	21
4.5 评价指标	23
4.5.1 主观评估	23
4.5.2 客观评估	23
4.6 实验结果	25
4.7 本章小结	26
第五章 结论和展望	29
5.1 总结	29
5.2 进一步工作	29

参考文献	31
致 谢	33

插图目录

1-1	An example of a buggy commit change in OPENSTACK	3
1-2	自动化代码提交信息生成流程图	4
3-1	LLM-MAA 方法流程图	16

表格目录

4-1	测试数据集统计表	21
4-2	基准方法与 LLM-MAA 方法指标对比	26

第一章 导论

1.1 研究背景

1.1.1 代码提交信息

代码提交信息 (Commit Message)^[1]是对代码变更内容 (what) 及原因 (why) 的一种自然语言描述, 它是对代码相关修改部分在内容和功能上的总结与概括。

代码提交信息对软件开发与维护至关重要。它能够帮助开发者快速了解代码变更的内容及原因, 提高开发和维护效率, 并且能够帮助开发团队快速了解其它模块的修改内容, 减少沟通成本, 提升协作效率, 使得团队间的协作变得方便快捷。简单易懂且准确的代码提交信息能够极大地降低开发者在软件开发和维护阶段的时间和精力成本, 这对于开发者来说非常重要。代码提交信息简明扼要地描述了代码更改的高层次意图, 帮助开发人员无需深入了解具体实现即可掌握软件的演变过程。它可以帮助开发者理解代码历史、追踪错误、协作开发, 提高代码的可读性和项目的可维护性。

1.1.2 人工代码提交信息生成

人工代码提交信息 (Manual Code Commit Message) 是指开发者在提交代码时, 通过人工描述的方式对代码变更内容、原因及影响进行书面概括介绍。它需要开发者主动思考, 并且需要开发者主动总结代码变更的内在逻辑。

人工生成代码提交信息需要确定规范框架, 对代码变更进行人工检查, 确定变更范围, 人工思考代码修改内容、解决了什么问题、以及代码变更的意义等, 并将上述思考结果用自然语言描述出来, 给出最终的代码提交信息结果。手动编写高质量的代码提交信息对开发者来说是额外的负担, 特别是在大型项目中, 这种负担尤为明显。

这种基于人工的代码提交信息生成存在诸多问题^[1]:

(1) 描述简略不完整且语义模糊

提交信息的描述质量高度依赖开发者的思考方式和对代码提交信息描述的规范性。人工生成提交信息难免会面临在繁杂的修改中无法全面捕捉代码变更全部内容的问题。并且，人工编写提交信息还可能导致对代码变更内容描述有偏差等问题。如图1-1所示的 OPENSTACK 示例中，虽然提交信息（line 4-10）指出了修改目的（“Name the securitygrouprules.direction enum”）和缺陷原因（“fixes Postgresql ENUM type requires a name errors”），但提交信息过于简略且语义模糊。

(2) 人工编写代码提交信息费时费力

人工编写代码提交信息需要开发者人工思考代码变更的内容及影响，并转换成易于人类理解的自然语言描述，这不仅耗费大量的时间，而且消耗开发者的精力。并且，代码变更通常涉及到广泛的代码语义变化，多模块间的关联也会导致单一的代码内容变更使得整体代码牵连过多，使得开发者人工编写提交信息变得困难。

(3) 与自动化工具整合困难

代码审查工具通常要求代码提交信息符合一定的规范，并且应当与上下文具有明显的关联性，但人工编写的信息可能不符合代码审查工具的上述自动化需求。

(4) 缺乏统一的结构与规范

人工编写的提交信息缺乏统一的结构，这不仅使得负责处理不同模块代码的开发者之间难以相互理解各自的代码提交信息，而且增加了自动化分析的难度。并且，自然语言的多样性导致缺乏统一的规范框架，这些都可能导致特征提取出现偏差。

(5) 难以兼顾代码变更的上下文导致描述有误

传统方法依赖人工设计的特征（如定位到代码修改的行、代码变更涉及到的模块等），但这些指标通常无法捕捉代码的真实语义。即使两组代码的量化特征完全相同，其行为也可能完全不同。例如，图1-1中的代码改动（line 16-28）需要结合提交信息才能理解其修复的是 ENUM 类型命名问题。

自动化代码提交信息生成技术能较好地应对上述问题。

```

1.      commit d60f6efd7f70efb1cccd007d55b1fa740fb98c76
2.      Author: Dan Prince <email address hidden>
3.      Date: Mon Jan 14 12:26:36 2013 -0500
4.      Name the securitygrouprules.direction enum.
5.      Updates to the SecurityGroupRule model and migration so that we
6.      explicitly name the securitygrouprules.direction enum. This fixes
7.      'Postgresql ENUM type requires a name.' errors.
8.
9.      Fixes LP Bug #1099267.
10.     Change-Id: Ia46fe8d4b0793caaabbfc71b7fa5f0cbb8c6d24b
11.     diff --git a/quantum/db/migration/alembic_migrations/versions/3cb5d900c5de
12.     _security_groups.py
13.     index ff39de84a..cf565af0f 100644
14.     --- a/quantum/db/migration/alembic_migrations/versions/3cb5d900c5de_
15.     security_groups.py
16.     +++ b/quantum/db/migration/alembic_migrations/versions/3cb5d900c5de_
17.     security_groups.py
18.     @@ -62,7 +62,10 @@ def upgrade(active_plugin=None, options=None):
19.     -         sa.Column('direction', sa.Enum('ingress', 'egress'), nullable=True),
20.     +         sa.Column('direction',
21.     +                     sa.Enum('ingress', 'egress',
22.     +                             name='securitygrouprules_direction'),
23.     +                     nullable=True),
24.     diff --git a/quantum/db/securitygroups_db.py b/quantum/db/securitygroups_db.py
25.     index 9903a6493..5bd890bbe 100644
26.     --- a/quantum/db/securitygroups_db.py
27.     +++ b/quantum/db/securitygroups_db.py
28.     @@ -62,7 +62,8 @@ class SecurityGroupRule(model_base.BASEV2, models_v2.HasId,
29.     -         direction = sa.Column(sa.Enum('ingress', 'egress'))
30.     +         direction = sa.Column(sa.Enum('ingress', 'egress',
31.     +                                     name='securitygrouprules_direction'))

```

图 1-1 An example of a buggy commit change in OPENSTACK

1.1.3 自动化代码提交信息生成

代码提交信息自动生成（Automatic Commit Message Generation, ACMG）技术^[2]是近年来软件开发与维护领域的重要研究方向之一，其核心目标是通过自动化方式为代码变更生成高质量的代码提交信息。ACMG 技术是指利用技术手段自动生成提交信息，以描述代码更改的内容（what）和原因（why）。利用此项技术，可以有效避免人工编写代码提交信息的描述简略且语义模糊、缺乏统一的结构与规范、难以兼顾代码变更的上下文导致描述有误、费时费力、与自动化工具整合困难等问题。在现代软件开发中，规范且准确的提交信息不仅能够使得开发者快速理解代码变更的内容（what）及原因（why），而且能够显著提升团队协作效率和项目可维护性。ACMG 技术可以快速、准确、规范地生成代码提交信息，准确捕捉到开发者更改代码的意图，其生成的提交信息更便于其他开发者理解。该技术将开发者从繁冗麻烦的编写代码变更信息的工作中解脱出来，显著提高了软件开发与维护的效率，帮助开发者精确了解代码变更的内容及原因。并且统一的提交信息规范可以省去整个开发团队对自然语言描述的差异化理解，提高团队协作效率。在团队开发中，提交信息的质量直接影响到代码审查和项目管理。自动化代码提交信息生成能够保持一定的规范性和一致性，提升团队成员

之间的沟通效率。

ACMG 技术通过自动化方式（如基于规则、检索、机器学习等方法）进行代码变更内容及原因的生成。

自动化代码提交信息生成（ACMG）的流程如图1-2所示：

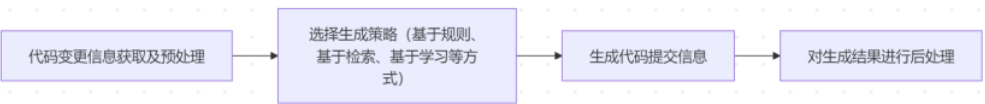


图 1-2 自动化代码提交信息生成流程图

ACMG 的流程通常分为四个阶段^[2]：

1. 在代码变更信息获取及预处理阶段，ACMG 技术从版本控制系统（如 Git）或集成化开发环境（IDE）中捕获代码变更差异数据（diff），并将这些 diff 描述为 json 格式的文本文件。随后，该技术会对比本地工作区与版本库的差异数据，提取包含增删改操作的代码片段。在预处理时，ACMG 技术会过滤掉非实质性的修改，进行深层解析，准确识别出涉及业务逻辑的关键修改位置。最后，该技术会根据修改性质建立分类体系，同步记录变更影响范围及上下文调用关系，形成具备完整语义特征的变更数据集，以供后续分析使用。

2. 在选择生成策略阶段，ACMG 技术会在整体上综合评估代码变更的深层次原因、历史提交数据特征、项目提交规范等来进行生成方法的选择。对于小型修改或重复模式变更优先采用基于规则模板的方法；对于逻辑变更复杂且涉及多样信息整合要求的代码变更优先采用基于检索的方法，利用相似历史提交案例优化生成质量；针对复杂逻辑变更则采用基于学习的方法，调用预训练代码模型（如 CodeBERT、CodeT5）进行语义推理与提交信息生成；此外，还可采用基于上述方法的混合方法，混合方法融合了规则、检索、学习的优势，通过检索上下文信息、对比规则模版等方式对代码提交信息进行自动化生成，该方法可以生成综合性的提交信息。

3. 生成代码提交信息时，基于规则的方法采用抽象语法树（AST）解析生成信息与预定义模板的匹配度，在识别代码变更中的方法异常处理、调用等模式后，基于规则的方法通过关键词提取生成代码提交信息；基于检索的方法利用向量化匹配方式从历史提交库中寻找相似代码变更案例，改写或复用与其关联的提交信息，此方法通常结合代码嵌入相似度或 TF-IDF 计算实现对上下文感知

的检索；基于学习的方法则采用早期机器学习技术对代码差异进行端到端生成，通过编码器解析代码语法结构与语义特征，通过解码器生成符合自然语言习惯的描述。随着深度学习技术的兴起，此方法还可融合强化学习机制以进一步优化生成结果，提高与人工提交风格的吻合度；混合方法综合考虑规则、检索、学习的优势，通过多维度对比与深度学习的智能推断来生成提交信息。

4. 对生成结果进行后处理时，ACMG 技术会提高文本的可读性和格式的规范性。在该阶段需要对生成的提交信息进行去除多余符号、抛弃多余的描述性语句等操作，以使得代码提交信息便于人类理解，同时便于机器处理。

1.2 研究现状

1.2.1 相关工作概述

根据现有研究，ACMG 技术可分为以下四类^[2]：

(1) 基于规则的方法：其核心思想是通过预定义的规则或模板来生成代码变更内容。基于规则的方法通过分析代码差异（diff），提取特定模式，并套用预定义的文本模板生成提交信息。典型工具有 DeltaDoc、hangeScribe 和 AutoSumCC。这种方法适合描述模式固定且简单的变更，并且规则明确，生成结果易于理解。但是，它的生成信息单一（仅能生成“做了什么”（what），无法解释变更原因（why））。此外，它的泛化能力差，对于复杂或非常规的代码变更（如重构），生成的提交信息可能冗长且不自然；并且其高度依赖人工设计，规则需针对不同语言或项目定制，维护成本高^[2]。

(2) 基于检索的方法：从历史提交中寻找相似代码差异（diff）并复用其代码提交信息。通过匹配历史提交中的相似代码变更直接复用或适配其信息。典型技术如 NNGen（使用最近邻算法检索相似提交生成简洁消息）、BM25 算法（基于词频 TF-IDF 快速匹配关键词）。这种方法的可解释性强，生成结果符合历史风格，但依赖高质量历史数据，对变更泛化能力弱^[3]。

(3) 基于学习的方法利用早期机器学习或深度学习模型从代码变更中学习语义特征，然后生成提交信息。神经机器翻译（NMT）模型如 NMTGen（这是一种基于 Seq2Seq 结构的模型，它使用注意力机制翻译代码变更，然后生成提交信息）、CommitBERT（这是一种以 CodeBERT 为底座的模型，它可以缩小编程语

言与自然语言的语义间隙)、PtrGNCMsg (该模型改进了 NMT 结构, 支持生成词汇表外的单词)。这种方法自动化程度高, 能够适应复杂语义的情况, 但需要大量训练数据, 并且生成结果可能不连贯^[3]。

(4) 混合方法结合了规则、检索与学习技术的优势, 可以增强生成质量。典型框架如 RACE (这是一种以检索相似提交作为示例的框架, 它利用神经模型对齐语义并引导生成)、CoRec (该框架是编码器-解码器架构, 它动态融合了检索结果的词分布概率)、ERICommiter (这是一种高效检索上下文学习的框架, 它通过两步过滤 (长度和语义) 优化检索库, 构建示例指导 LLM 生成)。这种方法融合了基于规则的方法的规范性、基于检索的方法的准确性以及基于学习的方法的泛化能力, 生成信息更全面 (包含明确全面的 “what” 和 “why” 且可满足指定的规范)^[3]。

1.2.2 现有工作的不足

对代码提交信息自动生成 (ACMG) 技术的研究还存在诸多局限性。上述方法存在代码变更内容 (what) 和原因 (why) 描述不准确、极度依赖已有提交信息数据集、可读性差等缺陷。随着大语言模型 (LLMs) 技术的兴起, 基于 LLMs 的 ACMG 技术实现在预测准确率、效率、成本、可读性等方面均超过上述方法。但是, 目前采用的基于 LLMs 生成代码提交信息的技术也存在无法准确输出代码变更内容 (what) 和原因 (why) 的问题。例如翻译式方法 (NMT) 生成的提交信息中 90% 存在重复、信息缺失问题^[4]。大多数方法仅生成单一句式的模式化信息, 缺乏解释代码变更目的 (“why”) 的上下文。

为解决在 ACMG 领域存在的上述技术缺陷, 本文提出了一种基于大语言模型的多 agent 方法 (LLM-MAA), 采用大模型进行检查反馈, 用以生成更加准确的、更加规范的、满足指定要求的代码提交信息。

1.3 研究思路

当前主流方法在生成提交信息时普遍存在内容不完整与信息维度缺失的问题, 基于深度学习的生成式模型虽提升了自然语言表达的流畅性, 但部分生成信息仍存在对代码修改意图 (why) 和具体变更内容 (what) 的生成偏差。

针对上述问题，本研究提出基于大语言模型的多 agent 方法（LLM-MAA）：利用大语言模型对生成的 commit message 进行检查，通过给出反馈意见来逐步完善 commit message。利用智能体 generator_agent 生成初始代码提交信息，利用智能体 reviewer_agent 对初始提交信息进行检查，并给出下一次迭代生成的反馈意见。如果反馈意见为“通过”，则终止下一次生成，直接进入结果后处理阶段；否则将反馈意见一并输入给 generator_agent，令其再次生成提交信息，进入下一次迭代循环。该方式可以动态实时检查代码提交信息（commit message）是否存在代码修改内容（what）或代码修改原因（why）不明确或缺失的问题，并在下一次迭代生成中得到改进。

1.4 本文主要贡献

本文的主要贡献包括：

（1）提出了一种基于大语言模型的多 agent 方法（LLM-MAA），利用大语言模型对生成的 commit message 进行检查，通过其给出的反馈意见来逐步完善 commit message。

（2）基于论文^[1]中的测试数据集与基准方法进行了对比实验，结果表明，在 BLEU、METEOR、ROUGE-L 指标上，LLM-MAA 方法均取得了优于基准方法的评分。在基座模型和 diff 数据集以及 prompt 均相同的情况下，本方法可以较基准方法取得更好的效果。同时在实际代码开发工作中，也能为代码提交信息生成工作进行补充参考。

1.5 论文组织

本文的剩余内容将按照如下格式组织：

第二章中，对各种代码提交信息自动生成方法的研究现状进行了详细描述。

第三章中，对本文提出的 LLM-MAA 方法进行了预备知识的介绍，并进行了工作框架、工作流程的介绍。

第四章中，介绍了实验环境、测试集、基座模型选择，并且对工作效果进行了基于数据集的测试评估，对测试结果进行了分析与对比。

第五章中，进行了总结与展望。

第二章 相关工作

2.1 相关工作概述

对代码提交信息自动生成 (Automatic Commit Message Generation, ACMG)^[2] 技术的研究近年来不断发展，主要形成了四类方法：基于规则的方法、基于检索的方法、基于学习的方法、混合方法。

基于规则的方法通过预定义模板生成提交信息，虽然简单高效但是高度依赖人工设计，泛化能力有限；基于检索的方法利用历史提交数据库，通过代码变更相似性检索匹配的提交信息，其生成结果可读性较强，但受限于历史数据的覆盖范围与质量；基于学习的方法借助机器学习技术（如 seq2seq 模型、Transformer），将代码变更作为输入直接生成自然语言描述，能够捕捉复杂语义模式并支持端到端生成，但对大规模标注数据与模型训练成本要求较高；近年来，混合方法成为研究热点，通过融合检索、学习与规则的互补优势，在生成准确性、可解释性与适应性之间寻求平衡，进一步推动了 ACMG 技术的实用化进程，但其仍存在实现难度大、复杂性较高、生成结果不准确、信息缺失等问题。

2.2 基于规则的方法

基于规则的方法 (Rule-based Approaches) 是代码提交信息自动生成 (ACMG) 领域最早采用的技术，其核心思想是通过人工设计的逻辑规则与结构化模板将代码变更特征映射为自然语言描述，使用预定义的模板来生成代码提交信息。这些方法能够生成满足特定格式的提交信息，但是通常无法充分描述代码变更的内容 (what) 和目的 (why)，且不能适应所有的提交信息生成情况。早期的研究通常采用预定义的规则和模板来生成代码提交信息，但是当遇到无法匹配模版的信息时，基于规则的方法可能不能生成准确的代码提交信息，并且该方法需要人工定义模板，缺乏统一的规范。

该方法的工作流程大致包括解析代码变更、语法与词法分析、解析变更类

型、解析变更粒度、分析变更的影响范围、进行规则模板选择等。该方法最初以简单模板匹配为核心，通过正则表达式提取代码变更中的文件名、函数名或关键词，将上述内容拼接成提交信息。但这样生成的结果缺乏语义且机械化。后来，研究者引入了自然语言处理启发式规则。改进后的方法根据代码操作类型针对性地选择动词，基于代码变更类型生成固定的句式，并将上述流程集成到 Git 等工具中以实现自动化。近年来，该方法进一步结合代码结构分析技术，利用 AST 提取并解析修改代码块的语义信息，基于上下文动态调整规则权重和代码修改频率。基于规则的方法透明可控、实现简单，但高度依赖人工设计规则，难以应对复杂语义场景^[2]。

2.3 基于检索的方法

基于检索的方法（Retrieval-based Approaches）通过从已有的代码提交信息中检索相似的代码变更来生成提交信息。然而，如果数据集中没有相似的代码变更，这种方法将会产生不准确的结果。该方法是一种类比生成范式，其核心逻辑是通过从历史提交信息库中检索与当前代码变更最相似的例子，复用或适配其对应的提交信息。该方法依赖于两个核心假设，即相似代码变更具有相似的提交描述，历史提交信息能够覆盖足够多的代码变更种类。

基于检索的方法在早期通过代码差异相似性匹配历史提交信息，它将代码变更表示为词袋（Bag-of-Words）或 TF-IDF 向量，随后利用余弦相似度检索相似提交信息（如 Genie）。2010 年代中期，信息检索技术被引入，该技术可以挖掘代码变更的深层意图。同时 Word2Vec 等语义向量化技术也进一步提升了基于检索的方法的匹配能力。这类方法依赖高质量历史数据，生成结果可读性较高，但通常伴随着数据稀疏性问题，对代码模式适应性较差，且这类方法高度依赖已有的提交信息库，对于新产生的代码变更难以生成恰当的描述^[3]。

2.4 基于学习的方法

基于学习的方法（Learning-based Approaches）利用机器学习模型（如早期机器学习方式）来生成代码提交信息。主要通过预训练模型和神经网络架构来实现。尽管此类方法在生成提交信息方面较规则、检索方法表现出色，但它们倾

向于生成常见的词汇，导致输出学术性、准确性、可读性较差，并且存在暴露偏差（exposure bias）等问题。

基于学习的方法在 2010 年代中期借鉴机器翻译思想，将提交信息生成视为代码到文本的翻译任务，使用 Seq2Seq 模型（如 RNN/LSTM）和注意力机制对齐代码变更与文本。随着 Transformer 架构兴起，预训练模型（如 CodeBERT、CodeT5）通过大规模代码到文本的训练来提升生成质量，CommitBERT 等专用模型进一步优化领域适配。近年来，研究者利用代码结构特性融合 AST 或控制流图（CFG）的方法（如 Tree-LSTM、图神经网络、CoRec），采用编码器-解码器混合架构，在推理过程中结合检索结果，通过融合输入和检索到的代码差异的概率分布来生成提交信息，并引入对比学习增强生成区分度，来提高代码提交信息自动生成的质量。该方法还可以通过 Few-shot Learning 降低数据依赖。

研究表明，基于学习的方法在生成准确性和语义相关性方面表现优异。但是当训练数据质量较低时，该方法生成的提交信息也会不准确。并且被开发者广泛使用的 MCMD 数据集中的代码提交信息普遍缺乏关键的“what”和“why”内容。基于学习的方法支持端到端的复杂语义生成，但其依赖大规模标注数据，且生成结果可解释性较弱。这种方法虽然也可引入深度学习技术，但由于需要进行大规模模型训练，不能充分利用预训练模型中蕴含的丰富知识，生成质量仍有提升空间^[3]。

2.5 混合方法

混合方法（Hybrid Approaches）结合了规则、检索和学习方法的优点，通过多模态技术融合与分层决策机制，结合上述三种方法的优势，以达到更好的生成效果。其设计遵循以下原则：

1. 优势互补：利用规则确保规范性、检索增强语义相关性、学习模型提升生成灵活性。
2. 知识蒸馏：将历史提交模式编码为可复用的启发式规则。
3. 动态路由：根据代码变更的复杂度自适应选择生成路径。

混合方法早期以规则、检索与学习模块的简单拼接为主，例如用规则生成候选提交信息后通过检索优化，或对模型输出进行规则校验。2020 年代初期，检

检索增强生成（RAG）^[5]框架兴起，其通过两阶段流程（先检索相似提交，再融合生成）结合数据与模型优势，并且还通过加入知识蒸馏技术（如 Dual Learning）进一步推动跨模块协作以生成高质量的代码提交信息。近年来，动态混合架构成为主流，例如自适应路由机制根据代码变更复杂度选择生成策略，或通过强化学习（如 RLHF）迭代优化多模块结果（如 Hybrid-CMG）。混合方法在 ACMG 领域卓有成效，但其仍存在高度依赖训练数据、生成过程过于复杂、部分缺失代码变更内容（what）及原因（why）等问题^[3]。

2.6 本章小结

本章系统地梳理了代码提交信息自动生成（ACMG）技术的四类核心方法及其技术发展路径。基于规则的方法利用预定义的规则和模板可以实现高效生成提交信息，但其高度依赖人工设计且泛化能力受限；基于检索的方法利用历史数据相似性匹配生成可读性高的结果，但当数据质量较低、历史数据覆盖范围较窄时，其也不能生成高质量的提交信息；基于学习的方法借助机器学习技术捕捉复杂语义模式，能够实现端到端生成，但其对标注数据与算力要求较高，且可解释性不足；混合方法通过融合规则、检索与学习的优势，能够生成综合性的提交信息，成为了当前的主流研究方向。同时，混合方法利用检索增强生成（RAG）与动态路由机制显著提升了生成质量与实用性，但其仍存在部分生成结果缺失代码变更内容（what）及原因（why）等问题。总体而言，ACMG 技术经历了系统化且逐步精进的过程，但目前仍有进步空间。

第三章 基于大语言模型的代码提交信息自动生成方法

3.1 预备知识

3.1.1 自然语言处理（NLP）

自然语言处理（Natural Language Processing, NLP）^[6]是指通过技术手段使计算机能够理解、生成和处理人类自然语言。其核心任务包括语义理解（识别词语和文本的深层含义）、词法分析、句法分析、文本摘要、情感分析、机器翻译等，主要可以概括为自然语言生成（NLG）与自然语言理解（NLU）。

NLP 的技术发展经历了从早期基于统计和规则的方法到深度学习驱动的转变。传统 NLP 是基于规则和统计的方法（如隐马尔可夫模型、词袋模型）。而现代技术以深度学习为主导，利用循环神经网络（RNN）、长短时记忆网络（LSTM）、Transformer 架构（例如 GPT 系列、BERT 等）、词向量（如 Word2Vec、GloVe）、神经网络（如 RNN、LSTM）等技术解析语言的语法、语义及上下文关联，利用海量数据训练语言表征，显著提升了语义理解能力和自然语言生成能力。NLP 的关键技术包括预训练-微调范式（如 BERT 的 Masked Language Modeling）、词嵌入（Word2Vec、GloVe）、注意力机制以及多模态融合。其应用场景涵盖众多领域。

自然语言处理（NLP）技术在代码提交信息自动生成（ACMG）中发挥着重要作用，主要体现在通过语义分析和上下文理解将代码变更转化为可读性强的、结构化的提交信息。基于深度学习的 NLP 模型（如 Transformer 架构或 GPT 系列）能够解析代码差异，识别新增功能、已修复的 Bug 和重构模块等关键信息，并且能够结合规范模板自动生成符合格式要求的提交信息，可以减少人工编写的不准确性和主观性，避免人工编写的语义模糊、缺乏明确的“what”和“why”等问题。NLP 还可通过多模态信息融合技术整合代码上下文与历史提交记录，来进一步提升生成的代码提交信息的准确性和一致性。

3.1.2 大语言模型

大语言模型（LLMs）可用于辅助编写代码提交信息。其可以通过分析代码变更的语义和结构特征，自动生成符合开发者设定要求的提交信息，能够显著提高开发效率和协作效率。LLMs 是基于深度学习来进行代码提交信息自动生成（ACMG）的核心^[7]。

LLMs 以 Transformer 等架构为核心，首先对代码差异（diff）进行多维度解析，如变量命名、代码块逻辑关系、函数提取、API 调用链等语法特征，同时结合开发者历史提交数据、代码注释等上下文信息，构建包含代码增删改语义的向量化表示。

在训练阶段，LLMs 通过学习代码与自然语言描述之间的映射关系，生成既能准确概括代码变更内容（what）和原因（why）、又能遵循特定的生成规范的代码提交信息。

当前该技术应用广泛，已搭载于 GitHub Copilot 等工具。但是其在平衡生成信息的简洁性、准确性与可追溯性之间仍有提升空间，未来或可通过多模态学习技术进一步提高生成质量^[8]。

3.1.3 代码专用预训练模型

代码提交信息自动生成（ACMG）技术可依托于代码专用预训练模型（如 CodeBERT、CodeT5^[9]）。这种模型通过预训练掌握代码变更和自然语言之间的复杂关联，进而深度解析代码结构和语义的变化，实现精准描述生成。

CodeBERT 专用模型^[10]基于 Transformer 架构和自注意力机制，能够解析代码的语法逻辑、语义功能，并将其转化为符合人类表达习惯的自然语言描述。这种能力使得该模型能够识别代码变更的“表面修改”（what），还能够理解修改背后的原因（why），生成准确且有逻辑性的提交信息。该模型基于多模态预训练框架，联合了 AST，并利用控制流程图、代码文本学习技术等结构化特征，通过融合自然语言注释构建跨模态对齐的方式来提高代码提交信息的生成质量。

CodeT5 模型采用代码去噪自编码与标识符感知预测技术，能够捕捉代码增删改操作的内容（what）与核心意图（why）。在 ACMG 任务中，该模型对代码差异（diff）进行分层编码，利用图神经网络嵌入 AST 节点间的依赖关系，利用

对比学习和预训练策略（如 SimCSE）来对齐代码变更片段与历史提交信息。在生成阶段该模型则采用约束解码技术，结合项目术语库和提交模板控制输出格式，确保生成信息的准确性。该类模型还可以结合缺陷检测、代码补全等功能，利用多任务学习框架，进一步提高生成的提交信息的质量。

3.1.4 代码提交信息规范

代码提交信息（commit message）是指开发者在提交代码变更时撰写的用于记录代码变更内容和原因的自然语言描述。其可帮助开发者和团队成员理解代码变更的内容和意图，帮助追踪了解代码库的历史演变，提高代码开发的效率和可维护性。

好的代码提交信息应该具有以下特点^[1]：完整且准确地描述变更内容（what）、准确描述变更的动机及原因（why）、描述变更的背景和需求来源、遵循特定规范、简洁明了。

3.2 基于大语言模型的多 agent 方法

3.2.1 总体流程

总体流程如图3-1所示。整个流程的输入是代码变更（diff），输出是具有完整变更内容和变更原因描述的高质量提交信息。

diff 首先被生成智能体 `generator_agent` 所接收，其根据 diff 的内容生成一个提交信息初稿，然后将提交信息初稿输入给评审智能体 `reviewer_agent`，由其生成反馈结果；若反馈结果符合要求（`reviewer_prompt` 中的要求），则进入提交信息后处理阶段，否则将反馈结果输入给生成智能体 `generator_agent`，由其再次生成提交信息，进入下一轮迭代循环。经过多轮迭代，直到生成的提交信息满足要求，或达到最大迭代次数。

基座模型是两个智能体的核心，其负责生成提交信息和反馈结果。在实验中，通过调用 `silicon flow` 平台的 API 实现对基座模型的调用。

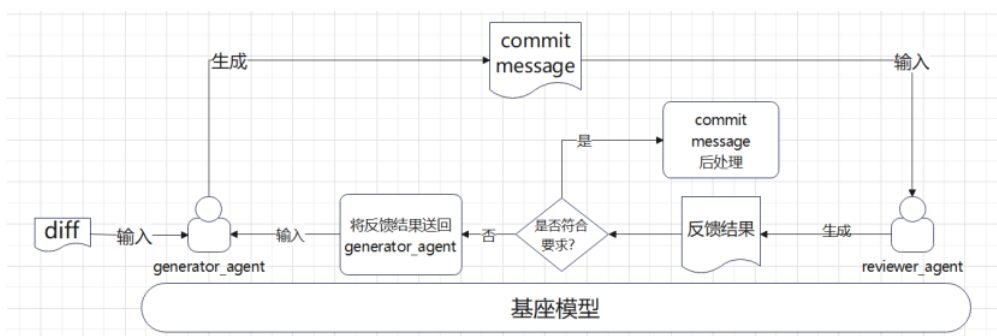


图 3-1 LLM-MAA 方法流程图

3.2.2 generator_agent 模块

该模块用于对输入进来的由 diff 构成的 prompt 进行 commit message 的生成。基座模型是 generator_agent 的核心，通过 prompt 的输入来生成提交信息。

具体过程为：输入 diff，将 generator_prompt 文本文件中的 DIFF 字段替换为 diff，构成 prompt，输入给 generator_agent 的基座模型，由其生成 commit message。

在 generator_prompt 的设计上，通过对基座模型进行模版研究，为 Python 语言选择了合适的提示：

“You are a programmer who makes the code changes. Please write a commit message for the code changes.

Please follow the following rule:

1.The commit message should include clear code modification content (what) and reasons for modification (why).

2.Only provide a commit message, do not add any other content.

3.Use professional and concise language to generate commit messages.

Git Diff

DIFF”

3.2.3 reviewer_agent 模块

该模块用于对输入进来的 commit message 进行反馈结果的生成。基座模型是 reviewer_agent 的核心，通过 prompt 的输入来生成反馈结果。

具体过程为：将 reviewer_prompt 文本文件中的 COMMIT MESSAGE 字段替换为生成的 commit message，构成 prompt，输入给 reviewer_agent 的基座模型，

由其生成审查反馈结果，检查 commit message 是否包含明确的 what 和 why，以及 what 和 why 是否完整，有无信息缺失等。若经 reviewer_agent 生成的审查反馈结果符合要求，则进入结果后处理阶段；否则将反馈结果送回 generator_agent 基座模型，进行下一轮迭代生成，再次生成 commit message，进行下一轮审查，依次迭代循环下去，直到生成的 commit message 满足要求（有明确的 what 和 why 且代码修改内容完整等）或达到人为设定的迭代次数上限即终止流程。

在提示选择上，reviewer_prompt 设计如下：

“You are a programmer who review the code commit message.

Please check if the commit message complies with the following specifications:

1.Include clear code modification content (what).

2.Include clear reasons for code modifications (why).

If the above requirements are not met, provide modification suggestions.

Commit to Review

COMMIT_MSG”

3.2.4 基座模型

基座模型是用于生成 commit message 和反馈结果的大语言模型。其内嵌于智能体 generator_agent 和 reviewer_agent 中。通过 silicon flow 平台的 API 来实现对基座模型的调用。

在此过程中，可以为 generator_agent LLM 和 reviewer_agent LLM 替换不同的基座模型，用以测定 LLM-MAA 方法在不同模型上的通用性。但是需要注意的是，本实验中要求 generator_agent 和 reviewer_agent 在每组实验中使用相同基座模型，以确保每组实验有单一变量。

3.2.5 结果处理

大语言模型生成的内容只是一段包含代码提交信息的描述性文本，其中往往含有多余的解释性语句、多余提示字符、多余符号等。在对结果进行分析之前，需要对生成序列进行处理，提取出有效内容并进行格式标准化，以作为完整的代码提交信息。

在结果处理工作中，采用如下处理方法：

去除多余的描述性语句；去除多余的解释性语句；去除生成序列中的多余字符；去除多余的句首句末空格；通过以上对代码提交信息语句进行的标准化操作，即可得到完整简洁的代码提交信息语句。

未经处理的 commit message 示例如下：

“The commit message is concise and accurately reflects both the code change and the reason behind it. It properly informs others who review the code about the addition of ‘volatile’ and its purpose in ensuring thread safety.\n\n**Final Commit Message:**\n\nAdd ‘volatile’ to ‘TestName.java’ field ‘name’ to ensure thread-safe access.”

可以看到，其中包含大量描述性语句及解释性语句。

经处理后的 commit message 如下：

“Add ‘volatile’ to ‘TestName.java’ field ‘name’ to ensure thread-safe access.”

可见，经处理后的 commit message 可读性更优，且更便于机器处理。

3.3 本章小结

在本章节中，对 ACMG 技术的相关知识（如 NLP，LLMs，代码专用预训练模型，以及代码提交信息规范等）进行了梳理与总结。并对 LLM-MAA 方法的整体流程进行了讲述。以及对智能体 generator_agent 和 reviewer_agent、基座模型、prompt、最终生成结果处理等方面进行了讨论。

第四章 实验与评估

4.1 实验环境

实验环境是本地代码 +siliconflow 远程 API 的组合，本地环境满足 API 调用条件（如 Python 版本、依赖库、网络策略），而模型计算实际由 siliconflow 的云端服务器执行。

1. 硬件配置：

-内存：16G

-CPU：AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz

2. 软件配置：

-操作系统：Windows 11

本地开发环境：

-VS Code

3. 模型托管平台：

-硅基流动（siliconflow）

4.API 客户端：

-调用 siliconflow API 的封装模块

4.2 基准方法选取

基准方法选取自论文^[11]中的方法：将 diff 输入给大语言模型，编写成 prompt 后利用大语言模型（LLMs）直接生成代码提交信息。

4.3 测试集选取

4.3.1 测试集获取

本次实验中，选用了论文^[1]中的测试数据集，共 271 个 diff 文件，以及对应的 commit messages。

首先从对应的 github 网站上用批处理程序下载每一个 diff 对应的 json 文件到本地，然后对这 271 个 diff 进行筛选，选出其中包含完整 what 和 why 的条目。最终得到 199 个 diff 文件，作为本次实验的最终测试数据集。

4.3.2 测试集处理

本次实验中，选用了论文^[1]中的测试数据集，共 271 个 diff 文件。从对应的 github 网站上用批处理程序下载全部 diff 对应的 json 文件到本地。

对上述 271 个 diff 进行分类后，可分为如下几类：

1. why 完整但 what 缺失。如：

“Fix dead link to the ant task in FAQ Documentation to the ant junit task has been moved to another location.”¹

2. what 完整但 why 缺失。如：

“Refactor: remove NetUtils.getHostAddress() and relace it with NetUtils.getLocalHost()
(<pr_link>) ”²

3. what 和 why 均完整。如：“Increase running time of statement. Fixes <issue_link> . <enter> The time between starting the statement and calling the future is arbitrary. Now the execution time of the statement is very very long. This ensures that the statement is not already finished when the future is called. ”³

筛选出其中 diff 对应的 commit message 包含完整 what 和 why 的条目。最终得到 199 个 diff 文件对应的条目，作为本次实验的最终测试数据集。测试数据集的统计表如下：

1 <https://github.com/junit-team/junit4/commit/24dfdd762e753c83babca3768d4b508cf5c6a86c>

2 <https://github.com/apache/dubbo/commit/fa157deadf65d00c9884d611430ec78e585840ff>

3 <https://github.com/junit-team/junit4/commit/1ef449139923a22333f262dc8c75fa057903040c>

表 4-1 测试数据集统计表

语言	测试集 diff 个数	平均大小	中位数大小	最大文件	最小文件
python	199	10990.41B	3217B	292702B	173B

4.4 基座模型选择

大模型根据源代码和权重是否公开可分为开源大模型（如 llama 系列）和闭源大模型（如 GPT 系列）。前者公开模型架构、训练代码和权重，允许自由修改、二次开发，后者仅通过 API 或有限接口访问，代码和权重保密。

根据有无推理功能可分为推理型大模型（如 DeepSeek-R1、GPT-4、Claude 3、DeepSeek-Math）和基础型大模型（如 GPT-3 早期版本、BERT、ChatGLM）。前者擅长逻辑推理、数学计算、多步骤问题解决等复杂任务。后者侧重文本生成或语义理解，但复杂推理能力较弱。

论文^[11]中的 ACMG 方法选用的基座模型为 ChatGPT（基于 OpenAI 的 GPT-3 模型，具体架构为 Transformer）。为公平起见，在本研究中基准方法使用的基座模型和 LLM-MAA 方法保持一致，选用如下基座模型：

本研究选用了开源推理大模型 DeepSeek-R1，以及开源无推理大模型 Qwen3 和 DeepSeek-V3 作为基座模型进行实验。同时，选用 DeepSeek-R1 的小参数版本以及 Qwen3 和 DeepSeek-V3 的大参数版本，用以测试其在 LLM-MAA 方法上的适用性以及进行横向对比。如下是对三个模型的介绍及其在 ACMG 领域的优势：

(1) DeepSeek-R1-Distill-Qwen-7B:

DeepSeek-R1-Distill-Qwen-7B 是基于 Qwen2.5-Math-7B 通过知识蒸馏得到的模型。该模型使用 DeepSeek-R1 生成的 80 万个精选样本进行微调，展现出优秀的推理能力。在多个基准测试中表现出色，其中在 MATH-500 上达到了 92.8% 的准确率，在 AIME 2024 上达到了 55.5% 的通过率，在 CodeForces 上获得了 1189 的评分，作为 7B 规模的模型展示了较强的数学和编程能力。¹

本研究选取 DeepSeek-R1 模型的小参数版本，用以探索本实验方法（LLM-MAA）在有推理功能的小参数模型上的适用性。

DeepSeek-R1-Distill-Qwen-7B 模型在代码提交信息自动生成（ACMG）领域

1 引用自 silicon flow 官网 [deepseek-ai/DeepSeek-R1-Distill-Qwen-7B](https://deepseek-ai.com/deepseek-r1-distill-qwen-7b) 的模型信息

优势明显。其能够利用推理功能自动分析代码变更差异，通过 RAG 技术结合项目历史提交数据，生成符合规范的提交信息。该模型继承了原版 DeepSeek-R1 的强推理能力，同时以 7B 参数量可以实现高效本地部署，能够精准识别代码修改的内容及逻辑，生成高精确度的代码提交信息。

(2)Qwen3-235B-A22B

Qwen3-235B-A22B 是通义千问系列的最新大语言模型，采用混合专家 (MoE) 架构，拥有 235B 总参数量和 22B 激活参数量。该模型独特地支持在思考模式（适用于复杂逻辑推理、数学和编程）和非思考模式（适用于高效的通用对话）之间无缝切换，显著增强了推理能力。模型在数学、代码生成和常识逻辑推理上表现优异，并在创意写作、角色扮演和多轮对话等方面展现出卓越的人类偏好对齐能力。此外，该模型支持 100 多种语言和方言，具备出色的多语言指令遵循和翻译能力。¹

本研究选取 Qwen3 模型的大参数版本，用以探索本实验方法（LLM-MAA）在无推理功能的大参数模型上的适用性。

Qwen3-235B-A22B 模型在代码提交信息自动生成（ACMG）领域性能良好，其能够精准理解代码变更逻辑并生成专业、简洁的提交信息。该模型能够结合对代码生成任务进行优化的训练数据，完整捕获代码库历史变更和关联文件的内容。其生成的代码描述能根据开发者偏好自动适配提交风格，而且具有良好的语法规范。

(3)DeepSeek-V3

新版 DeepSeek-V3（DeepSeek-V3-0324）与之前的 DeepSeek-V3-1226 使用同样的 base 模型，仅改进了后训练方法。新版 V3 模型借鉴 DeepSeek-R1 模型训练过程中所使用的强化学习技术，大幅提高了在推理类任务上的表现水平，在数学、代码类相关评测集上取得了超过 GPT-4.5 的得分成绩。此外该模型在工具调用、角色扮演、问答闲聊等方面也得到了一定幅度的能力提升。²

本研究选取 DeepSeek-V3 模型的大参数版本，用以探索本实验方法（LLM-MAA）在无推理功能的大参数模型上的适用性。

DeepSeek-V3 模型在代码提交信息自动生成（ACMG）领域能够精准识别代

1 引用自 silicon flow 官网 Qwen/Qwen3-235B-A22B 的模型信息

2 引用自 silicon flow 官网 Pro/deepseek-ai/DeepSeek-V3 的模型信息

码变更内容并生成符合规范的提交信息。其能够智能分析代码内容差异和上下文信息，并且具有混合专家架构（MoE）和动态参数激活功能，支持高度定制化提示词模板，使得开发者可灵活配置提交类型和影响范围，确保团队代码库风格统一。此外，DeepSeek-V3 的 API 调用成本远低于 GPT 等闭源模型，且支持 128K 上下文窗口，可同时处理多文件变更和长提交分析，能够显著降低人工校验成本。

4.5 评价指标

4.5.1 主观评估

在主观评价方面，经过人工审查，可以发现本次实验最终生成的 commit message 均含有明确且完整的代码变更内容（what）和代码变更原因（why），并且可读性良好，符合人类理解习惯。下面针对同一 diff 输入，列出利用 LLM-MAA 方法生成的 commit message、利用基准方法生成的 commit message 以及标准参考 commit message：

LLM-MAA：“Update security policy OkHttp version 4.x to be supported on Android 5.0+ and Java 8+, remove support for OkHttp version 3.14.x, and update OkHttp 3.12.x to Android 2.3+ and Java 7+ for better compatibility and wider support.”

（具有完整且明确的what和why）

基准方法：“Update version support dates and add new version, Modified 3.14.x, Added 3.12.x support”

（部分缺失what，完全缺失why）

标准参考：“Push OkHttp 3.14.x into the sea. It is no longer supported. (<pr_link>) <enter> OkHttp 4.x is our only branch receiving security fixes, bug fixes, and new features. <enter> OkHttp 3.12.x gets security fixes only. ”

（具有完整的what和why，但表述混杂，不够清晰）

4.5.2 客观评估

在客观评价方面，本次实验采用衡量生成文本与参考文本之间相似性的 BLEU-4、METEOR、ROUGE-L 指标。

(1)BLEU-4:

BLEU^[12]是用于评估生成的提交信息和参考提交信息差异的指标，取值在[0,1]，匹配度高就更接近1，反之更接近0。这个指标计算代价小，容易理解，与语言无关。BLEU 主要基于 n-gram 匹配（连续的 n 个词）的精确率进行指标计算，并结合短句惩罚（Brevity Penalty, BP）来调整长度差异的影响。

计算步骤：

n-gram 精确度（本研究中 n=4）：对 n=1,2,3,4 分别计算精确度（即生成文本与参考文本匹配的 n-gram 数量 / 生成文本中总 n-gram 数量）。

几何平均：对四个 n-gram 的精确度取几何平均。

短句惩罚（BP）：若候选文本长度（c）小于最接近的参考文本长度（r），则惩罚因子为 $\sqrt[r]{c}$ ，否则 BP=1。

$$BP = e^{1-\frac{r}{c}} \quad (4-1)$$

最终公式：4-2， $W_n=0.25$ 。

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^4 w_n \log p_n \right) \quad (4-2)$$

(2)METEOR:

METEOR（Metric for Evaluation of Translation with Explicit ORdering）^[13]是一种用于评估机器翻译或文本生成质量的自动指标，通过结合精确率、召回率及词序一致性来进行综合评估，进而克服 BLEU 等指标的局限性。METEOR 能够达到与人工量化判断高度一致，它能检测不同系统或同一系统版本的细微差异，具有一致性、可靠性和通用性。其实现了比传统指标更接近人类判断的评估效果，尤其在捕捉词义和词序差异方面表现突出。它首先通过多阶段匹配（表面形式匹配、词干匹配、同义词匹配）对齐候选文本与参考文本的词汇，得到匹配词数 m，并计算精确率 $P=m/c$ （c 为候选文本词数）和召回率 $R=m/r$ （r 为参考文本词数）。

然后采用调和平均值4-3作为基础分数，强调召回率的重要性。

$$F = \frac{10PR}{R + 9P} \quad (4-3)$$

然后，根据对齐后形成的连续块数 ch 计算词序惩罚项 $penalty_{4-4}$ ，以降低词序混乱的影响。

$$Penalty = \gamma \left(\frac{ch}{m} \right)^3 \quad (4-4)$$

最终 METEOR 得分计算公式为：

$$METEOR = F \cdot (1 - Penalty) \quad (4-5)$$

(3)ROUGE-L:

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation based on Longest Common Subsequence)^[14]是一种基于最长公共子序列(LCS)的自动评估指标,它可用于衡量生成文本与参考文本之间的相似度。其主要思想是找到两个序列共同拥有的最长子序列,这个子序列不要求连续,但要保持原有的顺序。ROUGE-L 不仅能识别出句子间词面的相似性,还能够识别出句子结构上的相似性。ROUGE-L 基于 LCS 长度计算召回率 (LCS 长度占参考文本长度的比例, Recall, R) 和精确率 (LCS 长度占候选文本长度的比例, Precision, P), 并计算召回率和精确率的调和平均值。

$$P = \frac{\text{LCS 长度}}{\text{候选文本长度}} \quad (4-6)$$

$$R = \frac{\text{LCS 长度}}{\text{参考文本长度}} \quad (4-7)$$

最终 ROUGE-L 指标值计算公式如下 (β 用于调节召回率和精确率的权重 (通常设为 $\beta \geq 1$ 以更重视召回率)):

$$ROUGE - L = \frac{(1 + \beta^2) \cdot P \cdot R}{R + \beta^2 \cdot P} \quad (4-8)$$

4.6 实验结果

通过对基于论文^[1]中的测试数据集与论文^[11]中的基准方法进行对比实验, 本次实验的 LLM-MAA 方法在 DeepSeek-R1、Qwen3、DeepSeek-V3 三个基座模

型上的 BLEU-4、METEOR、ROUGE-L 三个指标的值如下：

表 4-2 基准方法与 LLM-MAA 方法指标对比

	大语言模型	BLEU-4	METEOR	ROUGE-L
基准方法	DeepSeek-R1	0.1396	0.2605	0.2842
LLM-MAA 方法	DeepSeek-R1	0.1563	0.2740	0.2959
基准方法	Qwen3	0.1221	0.2036	0.2831
LLM-MAA 方法	Qwen3	0.1394	0.2471	0.2869
基准方法	DeepSeek-V3	0.1064	0.2175	0.2643
LLM-MAA 方法	DeepSeek-V3	0.1189	0.2374	0.2756

上述评分结果数据表明，LLM-MAA 在 DeepSeek-R1、Qwen3、DeepSeek-V3 三个基座模型上的三个指标上均好于基准方法。本研究采用的方法在代码提交信息自动生成（ACMG）技术上生成的 commit message 在与参考提交信息的匹配度上均优于基准方法。在数据集、prompt、基座模型均相同的情况下，LLM-MAA 方法理论上具有更大的使用优势。这对自动生成代码提交信息技术有贡献意义。

在三个基座模型的横向对比方面，可以看到，有推理功能的 DeepSeek-R1 大模型在三个指标评分上均优于另外两个无推理功能的大模型。可见，在 ACMG 领域，推理大模型对于生成包含精确 what 和 why 的代码提交信息更具优势。这说明了推理功能在自动化代码提交信息生成领域的重要性。

同时，值得注意的是，在本次实验中，同样作为无推理大模型，DeepSeek-V3 的表现要弱于 Qwen3。在对实验整体输出结果进行分析后，发现 DeepSeek-V3 最终生成的 commit message 集中由于达到迭代次数上限导致终止迭代循环的情况明显多于 Qwen3。推测这可能是因为 DeepSeek-V3 的参数规模较大、生成复杂度较高，致使其达到了迭代生成次数上限，最终提前终止。本研究认为这是导致出现上述情况的主要原因。同时观察到 DeepSeek-V3 的生成速度慢于 Qwen3，这可能是由于参数规模大导致生成过程复杂度提升，最终导致生成速率减缓^[15]。

4.7 本章小结

在本章节中，对 LLM-MAA 方法的实验环境、基座模型选取、测试集选取、评估指标进行了讲述。并对最终的实验结果数据进行了分析和横向对比。数据表明，本文提出的 LLM-MAA 方法在其它条件均相同的情况下生成的代码提交

信息的准确度、规范性均好于基准方法；并且有推理功能的大模型较无推理功能的大模型在 ACMG 领域性能更优。

第五章 结论和展望

5.1 总结

本文针对在代码提交信息自动生成（ACMG）领域存在的提交信息内容不完整、信息维度缺失、具体变更内容（what）和代码修改意图（why）的生成存在偏差等问题，创新性地提出了一种新的解决方案——基于大语言模型的多 agent 方法（LLM-MAA）。利用大语言模型对生成的提交信息进行检查，通过给出反馈意见来逐步完善提交信息。该方法可以动态实时检查代码提交信息是否存在修改内容（what）或修改原因（why）不明确或缺失的问题，并在下一次迭代生成中得到改进。并通过实验验证了该方法的有效性。

本文的主要贡献：

（1）提出了一种基于大语言模型的多 agent 方法（LLM-MAA），利用大语言模型对生成的 commit message 进行检查，通过其给出的反馈意见来逐步完善 commit message。

（2）基于论文^[1]中的测试数据集与基准方法进行了充分的对比实验，结果表明，在 BLEU、METEOR、ROUGE-L 指标上，LLM-MAA 方法均取得了优于基准方法的评分。实验结果表明，在基座模型和 diff 数据集以及 prompt 均相同的情况下，本方法可以较基准方法取得更好的效果。同时在实际代码开发工作中，也能为代码提交信息生成工作进行补充参考。

5.2 进一步工作

本文提出的方法以及实验仍存在不足，可以从以下方面进行改进：

（1）基于上下文学习技术^[16]，增强生成智能体 generator_agent 的能力。通过对代码变更位置上下文的分析进行更为精确的提交信息生成。

（2）选取更多的基座模型（如 Open AI 的闭源模型 ChatGPT 等），针对更多的编程语言测试集进行更充分的对比实验，进一步证明该方法的通用性。

本次实验研究对 ACMG 技术的进一步发展做出了部分贡献，相信在未来，ACMG 技术定能有更为成熟的技术实现和更加广阔的应用前景。

参考文献

- [1] TIAN Y, ZHANG Y, STOL K J, et al. What makes a good commit message?[C] // Proceedings of the 44th International Conference on Software Engineering. 2022: 2389-2401.
- [2] ZHANG Y, QIU Z, STOL K J, et al. Automatic commit message generation: A critical review and directions for future work[J]. IEEE Transactions on Software Engineering, 2024, 50(4): 816-835.
- [3] XUE P, WU L, YU Z, et al. Automated commit message generation with large language models: An empirical study and beyond[J]. IEEE Transactions on Software Engineering, 2024.
- [4] LI J, FARAGÓ D, PETROV C, et al. Only diff is not enough: Generating commit messages leveraging reasoning and action of large language model[J]. Proceedings of the ACM on Software Engineering, 2024, 1(FSE): 745-766.
- [5] FAN W, DING Y, NING L, et al. A survey on rag meeting llms: Towards retrieval-augmented large language models[C] // Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2024: 6491-6501.
- [6] LAVRINOVICS E, BISWAS R, BJERVA J, et al. Knowledge Graphs, Large Language Models, and Hallucinations: An NLP Perspective[J]. Journal of Web Semantics, 2025, 85: 100844.
- [7] LI D, JIANG B, HUANG L, et al. From generation to judgment: Opportunities and challenges of llm-as-a-judge[J]. ArXiv preprint arXiv:2411.16594, 2024.
- [8] ZHANG L, ZHAO J, WANG C, et al. Using large language models for commit message generation: A preliminary study[C] // 2024 IEEE International Confer-

- ence on Software Analysis, Evolution and Reengineering (SANER). 2024: 126-130.
- [9] WU Y, LI Y, YU S. Commit Message Generation via ChatGPT: How Far Are We?[C]//Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering. 2024: 124-129.
 - [10] JIANG S, LI J, ZONG H, et al. AiXcoder-7B: A Lightweight and Effective Large Language Model for Code Processing[J]. ArXiv preprint arXiv:2410.13187, 2024.
 - [11] LOPES C V, KLOTZMAN V I, MA I, et al. Commit messages in the age of large language models[J]. ArXiv preprint arXiv:2401.17622, 2024.
 - [12] PAPINENI K, ROUKOS S, WARD T, et al. Bleu: a method for automatic evaluation of machine translation[C]//Proceedings of the 40th annual meeting of the Association for Computational Linguistics. 2002: 311-318.
 - [13] BANERJEE S, LAVIE A. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments[C]//Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization. 2005: 65-72.
 - [14] LIN C Y. Rouge: A package for automatic evaluation of summaries[C]//Text summarization branches out. 2004: 74-81.
 - [15] HENIGHAN T, KAPLAN J, KATZ M, et al. Scaling laws for autoregressive generative modeling[J]. ArXiv preprint arXiv:2010.14701, 2020.
 - [16] HENDEL R, GEVA M, GLOBERSON A. In-context learning creates task vectors[J]. ArXiv preprint arXiv:2310.15916, 2023.

致 谢

感谢导师徐锋对我的指导与帮助，感谢计算机科学与技术系的老师们对我的栽培，感谢同学提供的知识讲解帮助。

